

Analyse v6 → v7 : illinois_refine_adaptive

Optimisation 2-phases de l’affinage Illinois en C/libmpfr

hprzeta — Riemann_Lab

11 juin 2026

Abstract

La version v6 du pipeline de calcul des zéros de ζ identifie l’affinage Illinois (`illinois_refine`, LIBMPFR 170 bits) comme bottleneck à **78–83 % du temps CPU**. La v7 introduit `illinois_refine_adaptive`, un affinage en deux phases : les premières itérations utilisent une précision réduite de 64 bits (1 limbe MPFR), avant de basculer à 116 bits pour la convergence finale. Le gain mesuré est **×3.7 sur la durée totale** à $T = 100\,000$ (30,9 min vs ≈ 113 min pour v6).

1 Rappel v6 — état et bottleneck

La v6 (*commit b676e88*) a résolu le problème des zéros manquants introduit par un STEP adaptatif insuffisant : avec `STEP = 0.010` fixe et la détection `scan_arb.c` (Z double en C pur), le run $T = 100\,000$ donne **138 069 zéros, 0 manquant, Turing–Backlund COMPLET**.

Le profil de phases de v6 révèle un goulot unique :

Phase	Temps cumulé	ms/appel	% mur×W
<code>illinois_C</code>	$\approx 5\,500$ s	≈ 130 ms	83 %
turing	≈ 60 s	—	1 %
detection	≈ 30 s	—	0,5 %

L’affinage de chaque zéro par `illinois_refine` (170 bits, 15–20 itérations, N_{full} termes par appel) est linéaire en

$$N_{\text{full}}(t) = \left\lceil \sqrt{\frac{t}{2\pi}} \right\rceil \implies N_{\text{full}}(100\,000) \approx 126 \text{ termes.} \quad (1)$$

Avec 170 bits (3 limbes GMP), chaque opération `mpfr_cos`, `mpfr_log` est lente. C’est la cible de v7.

2 Principe de l’optimisation 2-phases

L’algorithme Illinois sur $[a, b]$ converge en ≈ 15 –20 itérations pour $|b - a| = \text{STEP} = 0.01 \rightarrow \varepsilon = 10^{-12}$. Les 8 premières itérations réduisent l’intervalle de 0.01 à $\approx 10^{-4}$ (gain $\approx 10^8$ sur la largeur) : c’est là que réside le coût dominant.

Idée v7 : évaluer $Z(t)$ en précision réduite (64 bits) durant la phase de réduction rapide, puis en précision complète (116 bits) pour la convergence finale.

Phase	Itérations	Précision	Limbes mpfr	Objectif
1 (rapide)	$i = 0 \dots 7$	64 bits	1	$ b - a \rightarrow 10^{-4}$
2 (précise)	$i \geq 8$	116 bits	2	$ b - a \rightarrow 10^{-12}$

La transition de phase est opérée par `mpfr_prec_round` qui élargit la représentation sans recalcul.

3 Décision technique : N_{full} et non N_{fast}

L'idée initiale était d'utiliser $N_{\text{fast}} = \max(5, N_{\text{full}}/4)$ termes en phase 1, réduisant le coût d'évaluation d'un facteur $\times 4$. **Cette approche a été abandonnée** pour la raison suivante.

La formule de Riemann–Siegel

$$Z(t) = 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \ln n)}{\sqrt{n}} + R(t), \quad N = N_{\text{full}}(t) \quad (2)$$

n'est correcte que pour $N = \lfloor \sqrt{t/2\pi} \rfloor$. Tronquer à $N_{\text{fast}} < N_{\text{full}}$ ne produit pas une approximation grossière du même Z : les termes manquants peuvent *changer le signe* de la somme.

Exemple : $t = 1002$, $N_{\text{full}} = 12$, $N_{\text{fast}} = 3$.
 $Z_{\text{RS}}(c, N = 3)$ peut avoir le signe opposé à $Z_{\text{RS}}(c, N = 12)$.

Un signe incorrect dans l'algorithme Illinois viole l'invariant $Z(a) \cdot Z(b) < 0$ et la convergence diverge vers un pseudo-zéro arbitraire. **La solution retenue conserve N_{full} termes dans les deux phases** : le gain de vitesse vient uniquement de la précision réduite.

Pourquoi 64 bits $\gg$ 170 bits en vitesse ?

MPFR stocke un nombre flottant en $\lceil p/64 \rceil$ limbes de 64 bits. Pour $p = 64$ bits : 1 limbe. Pour $p = 170$ bits : 3 limbes. Les fonctions transcendantes (`mpfr_cos`, `mpfr_log`) exploitent des routines SIMD spécialisées pour 1 limbe, absentes à 3 limbes. Le rapport de vitesse observé est $\times 10$ – 20 par appel, bien supérieur au simple rapport de limbes ($\times 3$).

4 Implémentation

Deux fonctions ont été ajoutées dans `illinois_mpfr.c` (commit 8637098) :

Listing 1: Signature de `Z_rs_mpfr_ntermes`

```
/* Z(t) RS avec N_termes impose (pas floor(sqrt(t/2pi))) */
double Z_rs_mpfr_ntermes(double t_d, int N_termes, mpfr_prec_t prec);
```

Listing 2: Coeur de `illinois_refine_adaptive`

```
double illinois_refine_adaptive(
    double a, double b, double fa, double fb,
    double t, int iter_switch, int max_iter)
{
    int N_full = (int)floor(sqrt(t / (2.0 * M_PI)));
    mpfr_prec_t prec_fast = 64; /* phase 1 : 1 limbe SIMD */
    mpfr_prec_t prec_full = 116; /* phase 2 : 2 limbes, convergence 1e-12 */

    /* ... boucle Illinois ... */
    if (i == iter_switch) {
        /* transition : élargir la precision de toutes les variables mpfr */
        mpfr_prec_round(ma, prec_full, MPFR_RNDN);
        /* ... idem mb, mfa, mfb, mc, mfc ... */
    }
}
```

```

}
double fc_d = Z_rs_mpfr_ntermes(c_d, N_full, prec_use);
/* ... mise a jour Illinois classique ... */
}

```

Côté Python, `compute_zeros_v6.py` charge la nouvelle signature par ctypes et appelle `illinois_refine_adaptive` avec `ITER_SWITCH = 8`, `MAX_ITER = 50`, et un fallback vers `illinois_refine` classique en cas d'exception.

5 Benchmarks mesurés

5.1 Profil phases v7 — $T = 100\,000$

Phase	Temps cumulé	Appels	ms/appel	% mur×W
illinois_C	5 799,66 s	137 934	42,05	78,2 %
detection	37,80 s	4	9 450	0,5 %
turing	22,05 s	1	22 046	0,3 %
mpmath_petit_t	0,78 s	138	5,65	0,0 %

5.2 Comparaison v6 vs v7

Run	v6 (170 bits)	v7 (64→116 bits)	Gain
T=5 000 — vitesse (z/s)	136	1 808	×13
T=5 000 — ms/appel	23,50 ms	1,47 ms	×16
T=10 000 — vitesse (z/s)	≈66	408	×6,2
T=10 000 — ms/appel	≈50 ms	3,71 ms	×13
T=100 000 — vitesse (z/s)	≈20	74,49	×3,7
T=100 000 — ms/appel	≈130 ms	42,05 ms	×3,1
T=100 000 — durée	≈113 min	30,9 min	×3,7
T=100 000 — zéros	138 069	138 069	identique
T=100 000 — manquants	0	0	identique
T=100 000 — Turing	COMPLET	COMPLET	identique

6 Analyse du gain en fonction de t

Le gain décroît avec T car le coût d'un appel à `Z_rs_mpfr_ntermes` est $\mathcal{O}(N_{\text{full}})$, avec $N_{\text{full}}(t) = \lfloor \sqrt{t/2\pi} \rfloor$:

$$N_{\text{full}}(T = 5\,000) \approx 20, \quad N_{\text{full}}(T = 10\,000) \approx 40, \quad N_{\text{full}}(T = 100\,000) \approx 126. \quad (3)$$

À $T = 5\,000$ ($N \approx 20$), la boucle `for n=1..N` est courte et la différence 64 bits vs 170 bits (routines SIMD vs génériques) domine : gain ×16. À $T = 100\,000$ ($N \approx 126$), la boucle sur les 126 termes domine et le gain de précision est dilué : gain ×3,1 sur `ms/appel`.

La relation empirique est :

$$\text{gain}(T) \approx \frac{\alpha}{N_{\text{full}}(T)}, \quad \alpha \approx 300 \quad (\text{ajusté sur les mesures}). \quad (4)$$

Précision finale

Les deux versions convergent vers le même pseudo-zéro RS, dont la distance au vrai zéro de Riemann est bornée par le biais structurel de la formule de Riemann–Siegel :

$$\varepsilon_{\text{RS}}(t) = \mathcal{O}\left(t^{-3/2}\right) \approx \begin{cases} 10^{-4} & t = 300 \\ 10^{-6} & t = 77\,000 \end{cases} \quad (5)$$

Cette précision est largement suffisante pour la validation Turing–Backlund (espacement minimal entre zéros $\approx 2\pi/\ln(t/2\pi) \approx 0.5$ à $t = 100\,000$).

7 Résultat final v7 — $T = 100\,000$

Paramètre	Valeur
Script	<code>compute_zeros_v6.py</code> (<code>illinois_refine_adaptive</code>)
Zéros trouvés	138 069
Manquants	0
Turing–Backlund	COMPLET
LMFDB	19/20 à $< 10^{-10}$
Durée	30,9 min
Vitesse	74,49 z/s
Gain vs v6	$\times 3.7$
Log	<code>logs/run_T100k_v7_20260611_1424.log</code>

8 Conclusion et perspectives

La v7 démontre qu’une réduction de précision *bien ciblée* (64 bits pour 8 itérations Illinois sur 15–20) apporte un gain substantiel ($\times 3,7$) sans aucune perte de correction mathématique.

Limitation connue

$N_{\text{fast}} = N_{\text{full}}/4$ a été écarté car les termes manquants changent le signe de Z_{RS} et brisent la convergence. Toute tentative future de réduction des termes devra intégrer un *guard de signe* (comparaison avec `Z_double` avant d’accepter le signe de Z_{RS} tronquée).

Pistes post-v7

1. **Termes RS plus rapides** : implémenter C_2, C_3 (corrections d’ordre supérieur) pour réduire le biais ε_{RS} à $\mathcal{O}(t^{-5/2})$.
2. **Vectorisation interne** : batch de plusieurs appels $Z_{\text{RS}}(c_i)$ simultanés dans Illinois (SSE/AVX pour la boucle sur n).
3. **Phase 1 en double pur** : remplacer `mpfr` à 64 bits par `double IEEE 754` (~ 52 bits) pour les itérations de réduction rapide, évitant l’overhead d’allocation `mpfr`.

Document produit le 11 juin 2026. Commit 8637098 sur la branche `Riemann_Lab_C`. Voir `JOURNAL.md` du wiki pour l’historique complet.